

Лекція

Неблокуюча передача
повідомлень в MPI

Неблокуючий обмін повідомленнями

- Виконання програми не призупиняється для отримання чи відправлення даних. Такий спосіб при правильному використанні може зменшити втрату ефективності програми через повільну передачу повідомлень. Методи обміну повідомленнями починаються з літери I (Immediate).
- Метод `MPI_Irecv` не містить параметр `status`. Перевірити успішність передачі повідомлення та інформацію у вигляді `MPI_Status` можна методом `MPI_Test`.
- Додатковий параметр `request` методів неблокуючого обміну повідомленнями означає дескриптор операції. Цей параметр можна передати методам `MPI_Wait` або `MPI_Waitall`, щоб примусити програму дочекатись завершення операції обміну повідомленнями.
- Усі буфери, що використовуються в неблокуючих обмінах повідомленнями, стають недоступними користувачу після запуску операції. Буфери даних стануть знову доступними після виконання методів `MPI_Wait` або `MPI_Waitall` для цих операцій обміну повідомленнями.
- !!! Вибір способу відправки повідомлень (блокуючий/неблокуючий) не впливає на вибір способу отримання. Повідомлення, які були відправлені **блокуючими** функціями, можна отримувати **неблокуючими** функціями, і навпаки.

Point to Point Communication: non-blocking send/receive

```
int MPI_Isend(    void *buf,
                  int count,
                  MPI_Datatype datatype,
                  int dest,
                  int tag,
                  MPI_Comm comm,
                  MPI_Request *request    );

int MPI_Irecv(    void *buf,
                  int count,
                  MPI_Datatype datatype,
                  int source,
                  int tag,
                  MPI_Comm comm,
                  MPI_Request *request    );

int MPI_Wait(     MPI_Request *request,
                  MPI_Status *status     );

int MPI_Waitall(  int count,
                  MPI_Request array_of_requests[],
                  MPI_Status array_of_statuses[]    );
```

Приклад: тестування неблокуючого обміну повідомленнями

```
#include <stdio.h>
#include <unistd.h>
#include <mpi.h>
const int MY_TAG = 42;
int main(int argc, char* argv[]){
    int rank;
    MPI_Init(&argc,&argv);
    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    if(rank == 0) {
        sleep(5);
        int data[] = { 1, 2, 3, 4, 5};
        MPI_Request send_req;
        MPI_Isend(&data, 5, MPI_INT, 1, MY_TAG, MPI_COMM_WORLD, &send_req);
        sleep(5); /* Процес 0 виконує дії, які не змінюють data */
        MPI_Wait(&send_req, MPI_STATUS_IGNORE);
    }
    if(rank == 1){
        int data[5];
        MPI_Request recv_req;
        MPI_Irecv(&data, 5, MPI_INT, 0, MY_TAG, MPI_COMM_WORLD, &recv_req);
        sleep(2); /* Процес 1 виконує дії, які не використовують data */
        MPI_Wait(&recv_req, MPI_STATUS_IGNORE);
        for(int i = 0; i < 5; i++) {
            printf(" %d", data[i]);
        }
    }
    MPI_Finalize();
    return 0;
}
```

Інші методи неблокуючого обміну повідомленнями

- Використовуються літери B, S or R для позначення buffered, synchronous або ready режиму обміну повідомленнями:

```
int MPI_IBsend(), int MPI_IBrecv(),  
int MPI_ISsend(), int MPI_ISrecv(),  
int MPI_IRsend(), int MPI_IRrecv(),
```